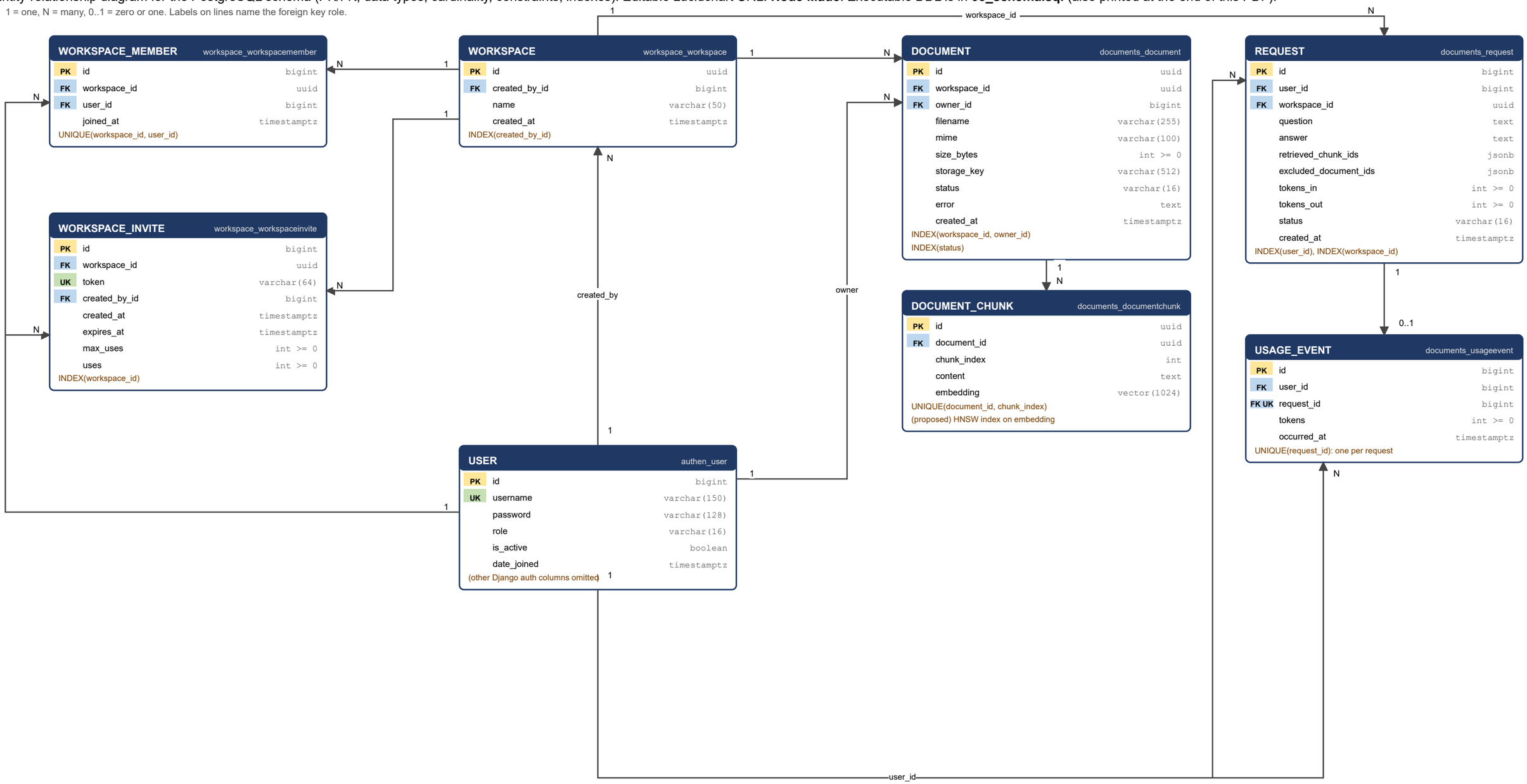
03 - Database Design: AI SaaS Workspace

Entity-relationship diagram for the PostgreSQL schema (PK/FK, data types, cardinality, constraints, indexes). Editable Lucidchart URL: **Node Made**. Executable DDL is in **03_schema.sql** (also printed at the end of this PDF).

1 = one, N = many, 0..1 = zero or one. Labels on lines name the foreign key role.



Data-integrity rules

Rule	How it is enforced	Where
Every row has a primary key	bigint identity for users, members, invites, requests, usage events; UUID for workspaces, documents, chunks (hard to guess)	PK constraints
Usernames are unique, ignoring case	Unique constraint plus a case-insensitive check before insert	UNIQUE(username) + serializer
A user joins a workspace once	Unique pair	UNIQUE(workspace_id, user_id)
Invite tokens cannot collide	Random 43-character URL-safe token	UNIQUE(token)
Invite counters stay valid	uses and max_uses cannot be negative; redeeming runs inside a transaction that locks the invite row so concurrent redemptions cannot exceed max_uses	CHECK >= 0 + SELECT FOR UPDATE
File ownership	Every document stores workspace and uploader; retrieval, delete and exclusion all compare owner_id with the caller; deleting a user is blocked while they own documents	FK owner_id (RESTRICT) + queries
Chunks cannot outlive their document	Deleting a document deletes its chunks	FK ON DELETE CASCADE
No duplicate chunk positions	One chunk per (document, index); re-ingestion deletes then inserts in one transaction (idempotent)	UNIQUE(document_id, chunk_index)
Request usage records	Each answered request has at most one usage event; users and requests cannot be deleted while usage rows exist (audit trail kept)	UNIQUE(request_id), FK RESTRICT
History survives document deletion	Requests keep chunk ids as JSON, not foreign keys; citations resolve to an empty list if the chunk is gone	jsonb retrieved_chunk_ids
Valid status values	Statuses are text choices checked in application code; database CHECK constraints are proposed	(proposed) CHECK
Tenant isolation	Every query filters by workspace and, for documents, by owner; non-members receive 404	Application queries

Indexes

Index	Table and columns	Purpose	Status
Primary keys and unique constraints	all tables	Lookups by id, token, username	Implemented
documents_document (workspace_id, owner_id)	documents_document	List and retrieve a user's documents in a workspace	Implemented
documents_document (status)	documents_document	Find pending/failed documents	Implemented
documents_documentchunk (document_id)	documents_documentchunk	Join chunks to documents	Implemented
Foreign key indexes on user_id / workspace_id	requests, usage, members, invites, workspaces	Join and filter performance	Implemented by the ORM
documents_usageevent (user_id, occurred_at)	documents_usageevent	30-day usage sum per user	Proposed
documents_request (workspace_id, user_id, created_at DESC)	documents_request	History list newest first	Proposed
HNSW on documents_documentchunk.embedding (vector_cosine_ops)	documents_documentchunk	Fast approximate nearest-neighbour search at scale	Proposed

Concurrency notes

Invite redemption is the one concurrent-write hotspot and is protected by a row lock. The daily token budget is checked before generation and recorded after, so simultaneous requests from one user can overshoot slightly (known gap; fix: reserve tokens atomically). This project has no booking/overlap constraints.

schema.sql (executable DDL, PostgreSQL 14+ with pgvector)

```
-- AI SaaS Workspace: database schema (PostgreSQL 14+ with pgvector) --
Derived by hand from the Django models (authen, workspace, documents).
-- Table/column names match Django's generated names so this file and the
-- migrations describe the same database.
--- PART 1 mirrors what the code defines today (see models.py files).
-- PART 2 is PROPOSED hardening that is NOT in the current migrations.
--
-- VERIFY BEFORE SUBMITTING: this file was written without running it against
-- a database. Apply it to an empty scratch DB, then compare with
```

```
--   pg_dump --schema-only <your migrated db>
-- and fix any differences.
--
-- NOTE on FK actions: Django enforces on_delete (CASCADE/PROTECT) in Python
-- and emits plain FKs. Here they are written as real ON DELETE clauses that
-- express the same intent (PROTECT -> RESTRICT).

CREATE EXTENSION IF NOT EXISTS vector; -- ---------------------------------

------------------------------- PART 1 --

CREATE TABLE authen_user (     id          BIGINT GENERATED BY DEFAULT
AS IDENTITY  PRIMARY KEY,       password       VARCHAR(128) NOT NULL,
-- hashed by Django      last_login    TIMESTAMPTZ,      is_superuser
BOOLEAN      NOT NULL DEFAULT FALSE,     username     VARCHAR(150) NOT
NULL UNIQUE,        first_name     VARCHAR(150)  NOT  NULL  DEFAULT  '',
last_name        VARCHAR(150)  NOT  NULL  DEFAULT  '',          email
VARCHAR(254) NOT NULL DEFAULT '',      is_staff      BOOLEAN       NOT
NULL DEFAULT FALSE,      is_active     BOOLEAN       NOT NULL DEFAULT
TRUE,     date_joined  TIMESTAMPTZ  NOT NULL DEFAULT now(),     role
VARCHAR(16)  NOT NULL DEFAULT 'member' );

CREATE TABLE workspace_workspace (     id          UUID PRIMARY KEY,
-- uuid4 generated by app     created_by_id BIGINT NOT NULL REFERENCES
authen_user(id) ON DELETE RESTRICT,     name        VARCHAR(50)  NOT NULL
DEFAULT 'Untitled Workspace',     created_at  TIMESTAMPTZ  NOT NULL DEFAULT
now() );
CREATE INDEX workspace_workspace_created_by_idx ON workspace_workspace (created_by_id);

CREATE TABLE workspace_workspacemember (     id           BIGINT GENERATED BY
DEFAULT AS IDENTITY PRIMARY KEY,     workspace_id UUID   NOT NULL REFERENCES
workspace_workspace(id) ON DELETE CASCADE,     user_id      BIGINT NOT NULL
REFERENCES authen_user(id)          ON DELETE CASCADE,     joined_at    TIMESTAMPTZ
NOT NULL DEFAULT now(),     CONSTRAINT workspace_member_unique UNIQUE (workspace_id,
user_id)
);
CREATE INDEX workspace_member_user_idx ON workspace_workspacemember (user_id);

CREATE TABLE workspace_workspaceinvite (     id           BIGINT GENERATED BY DEFAULT
AS IDENTITY PRIMARY KEY,     workspace_id UUID   NOT NULL REFERENCES
workspace_workspace(id) ON DELETE CASCADE,     token        VARCHAR(64) NOT NULL
  UNIQUE,      -- secrets.token_urlsafe(32)     created_by_id BIGINT NOT NULL REFERENCES
  authen_user(id) ON DELETE RESTRICT,     created_at   TIMESTAMPTZ NOT NULL DEFAULT
  now(),     expires_at   TIMESTAMPTZ NOT NULL,     max_uses     INTEGER NOT NULL
  DEFAULT 1 CHECK (max_uses >= 0),     uses         INTEGER NOT NULL DEFAULT 0 CHECK
  (uses >= 0) ); CREATE INDEX workspace_invite_workspace_idx ON
  workspace_workspaceinvite (workspace_id);

  CREATE TABLE documents_document (     id           UUID PRIMARY KEY,     workspace_id
  UUID   NOT NULL REFERENCES workspace_workspace(id) ON DELETE CASCADE,     owner_id
  BIGINT NOT NULL REFERENCES authen_user(id)         ON DELETE RESTRICT,     filename
  VARCHAR(255) NOT NULL,
    mime         VARCHAR(100) NOT NULL,     size_bytes   INTEGER NOT
NULL CHECK (size_bytes >= 0),     storage_key  VARCHAR(512) NOT NULL,
-- path in object/file storage     status       VARCHAR(16)  NOT NULL
DEFAULT 'pending',   -- pending | ready | failed     created_at
TIMESTAMPTZ  NOT NULL DEFAULT now(),     error        TEXT NOT NULL
DEFAULT ''
);
CREATE INDEX documents_document_ws_owner_idx ON documents_document (workspace_id,
owner_id);
CREATE INDEX documents_document_status_idx   ON documents_document (status);

CREATE TABLE documents_documentchunk (     id          UUID
PRIMARY KEY,     document_id UUID NOT NULL REFERENCES
documents_document(id) ON DELETE CASCADE,     chunk_index
INTEGER NOT NULL,     content     TEXT NOT NULL,     embedding
vector(1024) NOT NULL,     CONSTRAINT
documents_chunk_doc_index_unique UNIQUE (document_id,
chunk_index)
); CREATE INDEX documents_chunk_document_idx ON
documents_documentchunk (document_id);

CREATE TABLE documents_request (     id                   BIGINT GENERATED BY DEFAULT AS
IDENTITY PRIMARY KEY,     user_id               BIGINT NOT NULL REFERENCES
authen_user(id)         ON DELETE RESTRICT,     workspace_id          UUID   NOT NULL
REFERENCES workspace_workspace(id) ON DELETE CASCADE,     question              TEXT NOT
NULL,     answer                TEXT NOT NULL DEFAULT '',     retrieved_chunk_ids   JSONB
NOT NULL DEFAULT '[]',   -- soft refs; survive chunk deletion     excluded_document_ids
JSONB NOT NULL DEFAULT '[]',     tokens_in             INTEGER NOT NULL DEFAULT 0 CHECK
(tokens_in  >= 0),     tokens_out            INTEGER NOT NULL DEFAULT 0 CHECK (tokens_out
>= 0),     status                VARCHAR(16) NOT NULL DEFAULT 'pending',  -- pending |
success | failed | not_found     created_at            TIMESTAMPTZ NOT NULL DEFAULT now() );
CREATE INDEX documents_request_user_idx      ON documents_request (user_id);
CREATE INDEX documents_request_workspace_idx ON documents_request (workspace_id);

CREATE TABLE documents_usageevent (     id          BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY
KEY,     user_id     BIGINT NOT NULL REFERENCES authen_user(id)       ON DELETE RESTRICT,
request_id  BIGINT NOT NULL REFERENCES documents_request(id) ON DELETE RESTRICT,     tokens
INTEGER NOT NULL CHECK (tokens >= 0),     occurred_at TIMESTAMPTZ NOT NULL DEFAULT now(),
CONSTRAINT documents_usage_one_per_request UNIQUE (request_id)  -- at most one usage row per request
);
CREATE INDEX documents_usageevent_user_idx ON documents_usageevent (user_id);

-- ---------------------------------------------------------------- PART 2 --
-- PROPOSED (not implemented in the current migrations). Each item names the
-- problem it addresses.

-- Vector search currently scans every chunk matching the filters; add an ANN
-- index once data grows (HNSW, cosine to match the <=> operator in
retrieval).
-- CREATE INDEX documents_chunk_embedding_hnsw
--     ON documents_documentchunk USING hnsw (embedding vector_cosine_ops);

-- Usage check sums tokens per user over a 30-day window.
-- CREATE INDEX documents_usageevent_user_time_idx
--     ON documents_usageevent (user_id, occurred_at);

-- History list filters by (workspace, user) and sorts by newest.
-- CREATE INDEX documents_request_ws_user_created_idx
--     ON documents_request (workspace_id, user_id, created_at DESC);

-- Enforce enum values and invite counters at the database level.
-- ALTER TABLE documents_document ADD CONSTRAINT documents_document_status_chk
--     CHECK (status IN ('pending','ready','failed'));
-- ALTER TABLE documents_request ADD CONSTRAINT documents_request_status_chk
--     CHECK (status IN ('pending','success','failed','not_found'));
-- ALTER TABLE workspace_workspaceinvite ADD CONSTRAINT workspace_invite_uses_chk
--     CHECK (uses <= max_uses);
```